

ARTIFICIAL INTELLIGENCE

Lecture Set 02
Agents and Environments

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

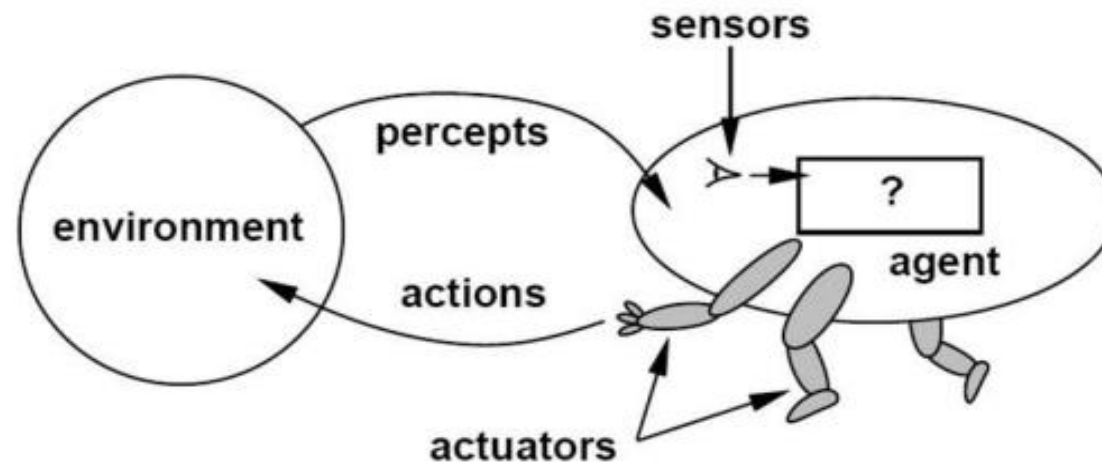
1. Introduction to AI
- **2. Agents and Environments**
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
6. Local Search
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Logical Agents
10. First-order Logic
11. Uncertain Knowledge and Reasoning
12. Supervised, Unsupervised, and Reinforcement Learning
13. Recent Trends in AI and Applications of AI Algorithms

Table of Contents

- Agent
- Rational Agents
- PEAS
- Environment
- Types of Environment
- Basic Agent Types

Agent

- An agent is anything that can be viewed as **perceiving** its environment through **sensors** and **acting** upon that environment through **actuators**.



The **agent function** maps from percept histories to actions:
 $[f: P^* \rightarrow A]$

The **agent program** runs on the physical **architecture** to produce f

Agent = Architecture + Program

Examples of Agent

■ Human Agent

- Sensors: Ears, Eyes, Tongue, Nose, and Skin.
- Actuators: Hands, Legs, Feet and Mouth

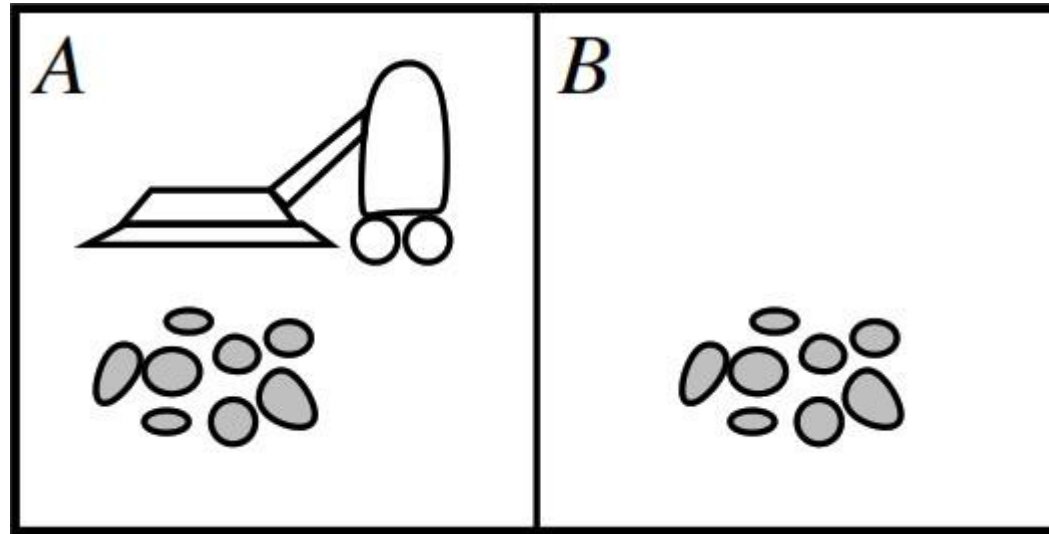
■ Robot

- Sensors: Cameras, Infrared range finders.
- Actuators: Motors, Mechanical hands and legs, Robotic arm.

■ Software Agent has encoded bit strings as its percepts and actions.

- **Chatterbots**: Agents that are used for chatting on the Web.
- **Spambots**: Agents that generate junk emails (spam) after collecting email addresses.
- **Mailbots**: Agents that manage and filter email (remove spam).

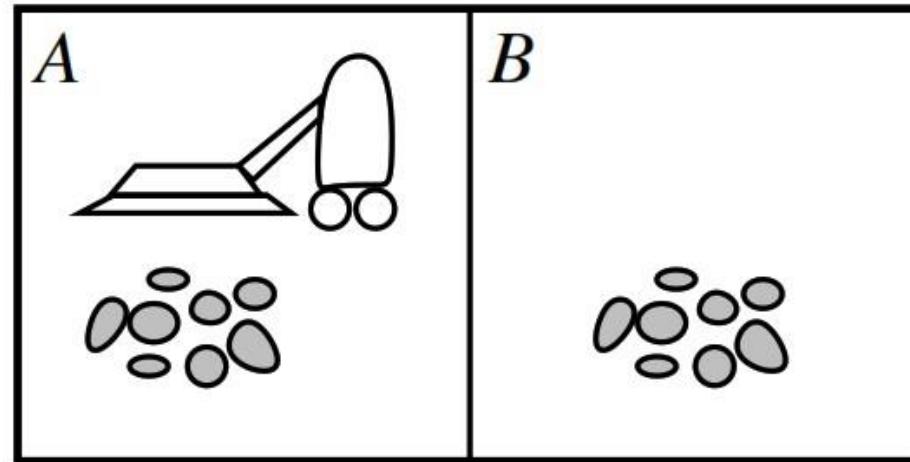
Vacuum-cleaner world



Percepts: location [A,B] and contents [Clean, Dirty]

Actions: *Left, Right, Suck, DoNothing*

A vacuum-cleaner Agent



Tabulation of an agent function of the vacuum-cleaner

Percept sequence	Action
$[A, Clean]$	<i>Right</i>
$[A, Dirty]$	<i>Suck</i>
$[B, Clean]$	<i>Left</i>
$[B, Dirty]$	<i>Suck</i>
$[A, Clean], [A, Clean]$	<i>Right</i>
$[A, Clean], [A, Dirty]$	<i>Suck</i>
$\vdots$	$\vdots$

function REFLEX-VACUUM-AGENT($[location, status]$) **returns** an action

if $status = Dirty$ **then return** *Suck*
else if $location = A$ **then return** *Right*
else if $location = B$ **then return** *Left*

Rational Agents

An agent should strive to "do the right thing", based on what it can perceive and the actions it can perform. The right action is the one that will cause the agent to be most successful.

Performance measure: an **objective criterion** for success of an agent's behavior.

E.g., performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.

Rational Agents

Rational Agent: For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

PEAS

When designing a rational/intelligent agent, we keep in mind PEAS.

PEAS: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors

Consider, e.g., the task of designing an automated taxi driver:

- Performance measure
- Environment
- Actuators
- Sensors

PEAS

Agent: automated taxi driver

- Performance measure: *Safe, fast, legal, comfortable trip, maximize profits*
- Environment: *Roads, other traffic, people and objects in/around the street*
- Actuators: *Steering wheel, accelerator, brake, signal, horn*
- Sensors: *Cameras, sonar, speedometer, GPS, odometer, engine sensors, keyboard*

PEAS

Agent: Part-picking robot

- Performance measure: Percentage of parts in correct bins
- Environment: Conveyor belt with parts, bins
- Actuators: Jointed arm and hand
- Sensors: Camera, joint angle sensors



PEAS

Agent: Interactive English tutor

- Performance measure: Maximize student's score on test
- Environment: Set of students
- Actuators: Screen display (exercises, suggestions, corrections)
- Sensors: Keyboard

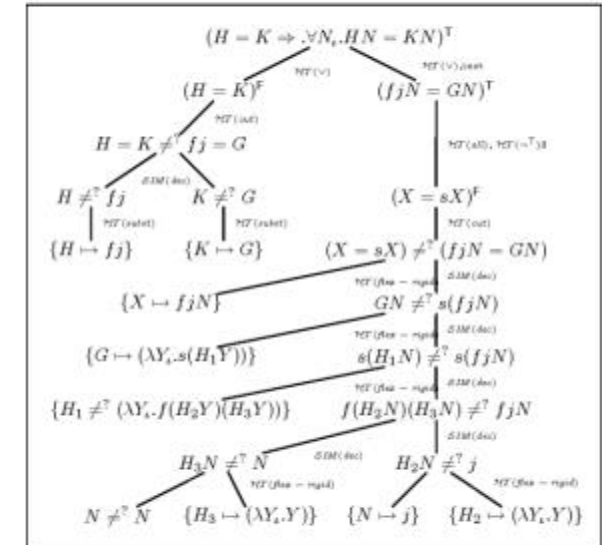
Develop PEAS description of the followings



Robot Soccer Player



Internet Book-Shopping Agent



Theorem-Proving Assistant

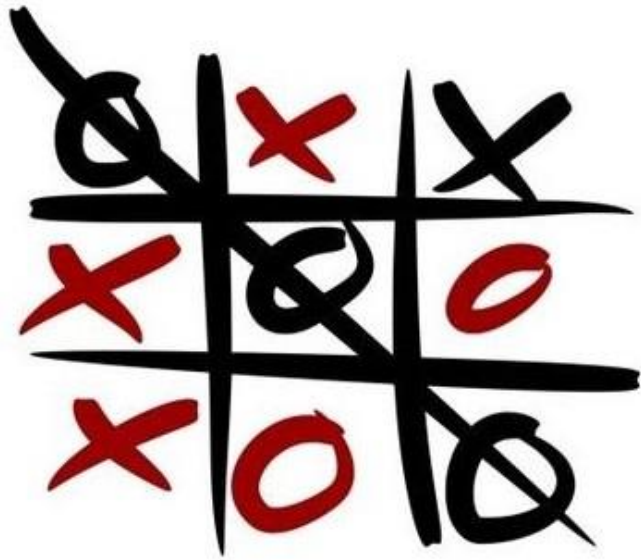
Environment

- An environment is everything that surrounds the agent that is not the part of agent itself.
- This is where agent lives or operates.
- It provide the agent with something to sense and somewhere to move around it.

Types of Environment

1. Fully observable / Partially observable:

- If agent has access to entire state of the environment through sensor, it is fully observable.
- If part of the environment is observable, it is partially observable



Tic Tac Toe is Fully Observable.

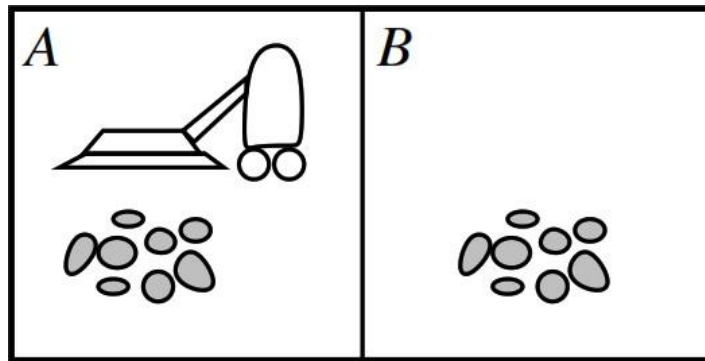


Cards are Partially Observable.

Types of Environment Cont'd

2. Deterministic/Stochastic:

- An environment is deterministic if the next state of the environment is completely determined by the current state of the environment and the action of the agent.
- In a stochastic environment, there are multiple, unpredictable outcomes.
- In a fully observable, deterministic environment, the agent need not deal with uncertainty.



Vacuum is deterministic.



Ludo is stochastic.

Types of Environment Cont'd

3. Episodic/Sequential:

- The environment is episodic if each of the agent's tasks does not rely on past performance or cannot affect future performance.
- If not, then it is sequential.



Robot is Episodic.

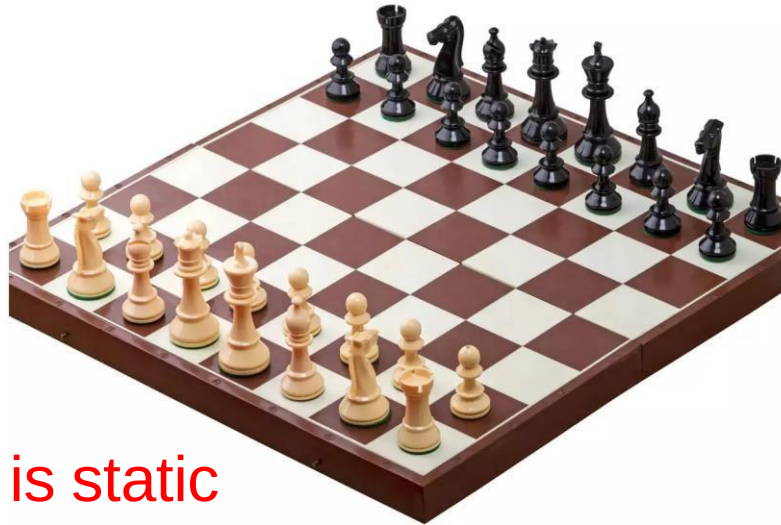


Taxi driver is sequential.

Types of Environment Cont'd

4. Static/Dynamic:

- A static environment does not change while the agent is thinking.
- The passage of time as an agent deliberates is irrelevant.
- The agent doesn't need to observe the world during deliberation.
- If the environment can change while an agent is deliberating, then we say the environment is dynamic for the agent.



Chess is static

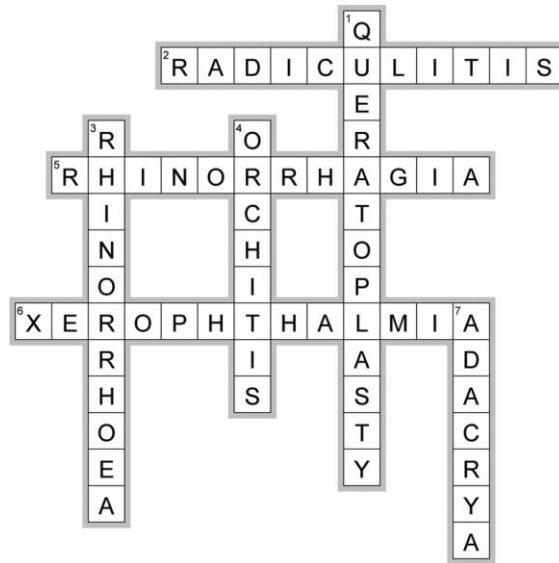


Taxi driver is dynamic

Types of Environment Cont'd

5. Single-Agent/Multi-Agent:

- In multi-agent environment, there are other intelligent agents as well which are either cooperative or competitive agents.
- In single-agent, there is only one agent working in the environment.



Crossword is Single agent, while Chess is a multi-agent environment.

Types of Environment Cont'd

6. Discrete/Continuous:

- Discrete has finite number of possible states.
- Number of states in continuous environment is infinite.



Chess has a finite number of distinct states. thus it is discrete;
however the Taxi-driving is not.

Example: Environment Types

Task Environment	Oberservable	Deterministic	Episodic	Static	Discrete	Agents
<i>Crossword puzzle</i>						
<i>Chess with a clock</i>						
<i>Taxi driver</i>						
<i>mushroom-picking</i>						

- The **environment type** largely determines the **agent design**
- The **real world** is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

Example: Environment Types

Task Environment	Oberservable	Deterministic	Episodic	Static	Discrete	Agents
<i>Crossword puzzle</i>	fully	deterministic	sequential	static	discrete	single
<i>Chess with a clock</i>	fully	strategic	sequential	semi	discrete	multi
<i>Taxi driver</i>	partially	stochastic	sequential	dynamic	conti.	multi
<i>mushroom-picking</i>	partially	stochastic	episodic	dynamic	conti.	single

- The **environment type** largely determines the **agent design**
- The **real world** is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

Basic Agent Types

Agents can be grouped into six classes based on their degree of perceived intelligence and capacity.

1. **Table Driven Agents** (look-up table of percept-action pairs)
2. **Simple Reflex Agents** (present condition-based decision)
3. **Model Based Reflex Agents** (memory-based decision)
4. **Goal Based Agents**
5. **Utility Based Agents** (Maximize the profit)
6. **Learning Based Agents** (learn from experience)

Table Driven Agents

- They are implemented by a (large) lookup table.
- Table lookup of percept-action pairs mapping from every possible perceived state to the optimal action for that state.

Problems:

- *Too big to generate and to store (Chess has about 10^{120} states).*
- *No knowledge of non-perceptual parts of the current state.*
- *Not adaptive to changes in the environment; requires entire table to be updated if changes occur.*
- *Can't make actions conditional on previous actions/states.*

Table Driven Agents

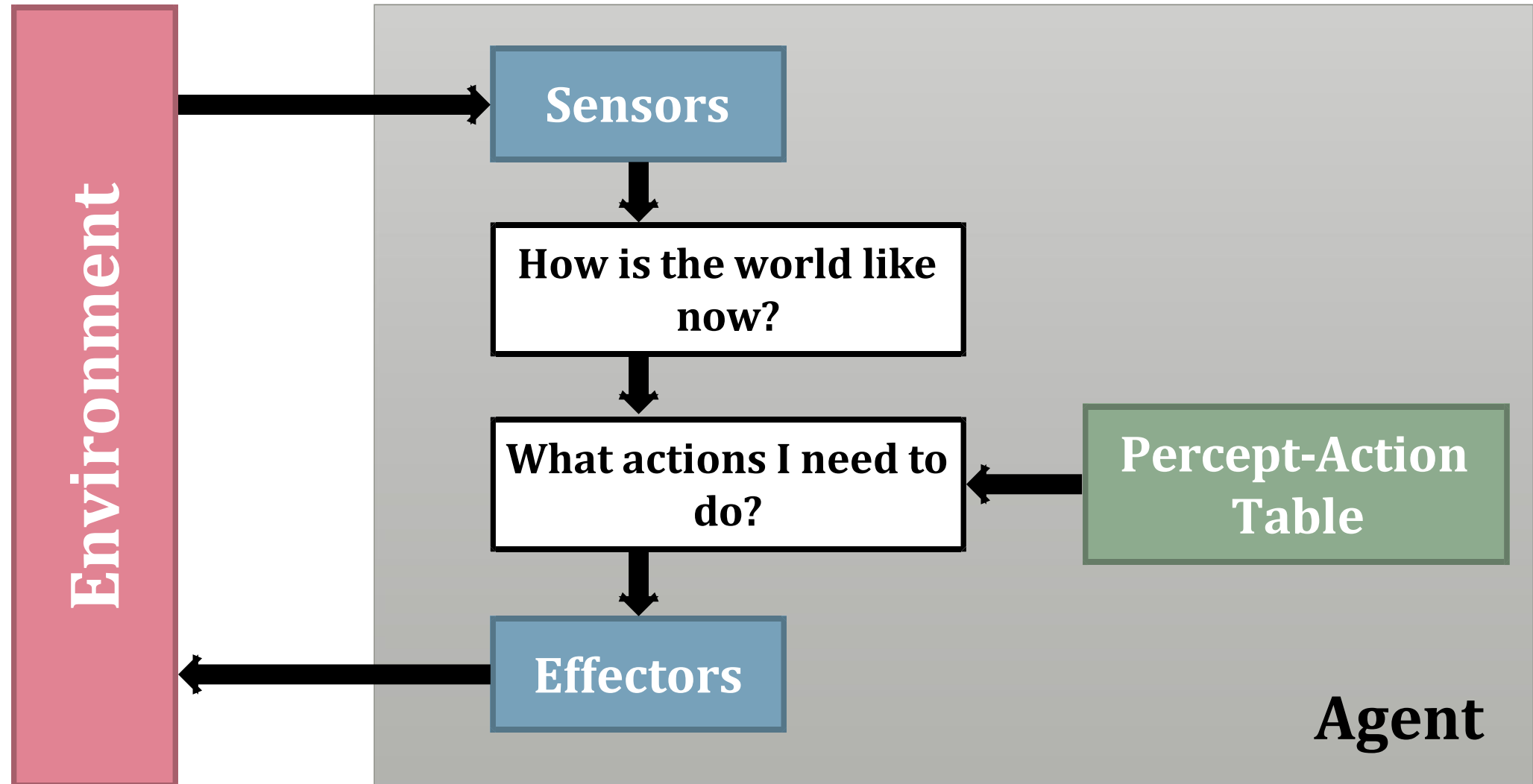


Table Driven Agents

For Automated Taxi Driving Agent:

- The option of constructing an explicit lookup table is out of the question.
- The visual input from a single camera comes in at the rate of 50 megabytes per second .
- So the lookup table for an hour would be $2^{60} \times 60 \times 50M$ entries.

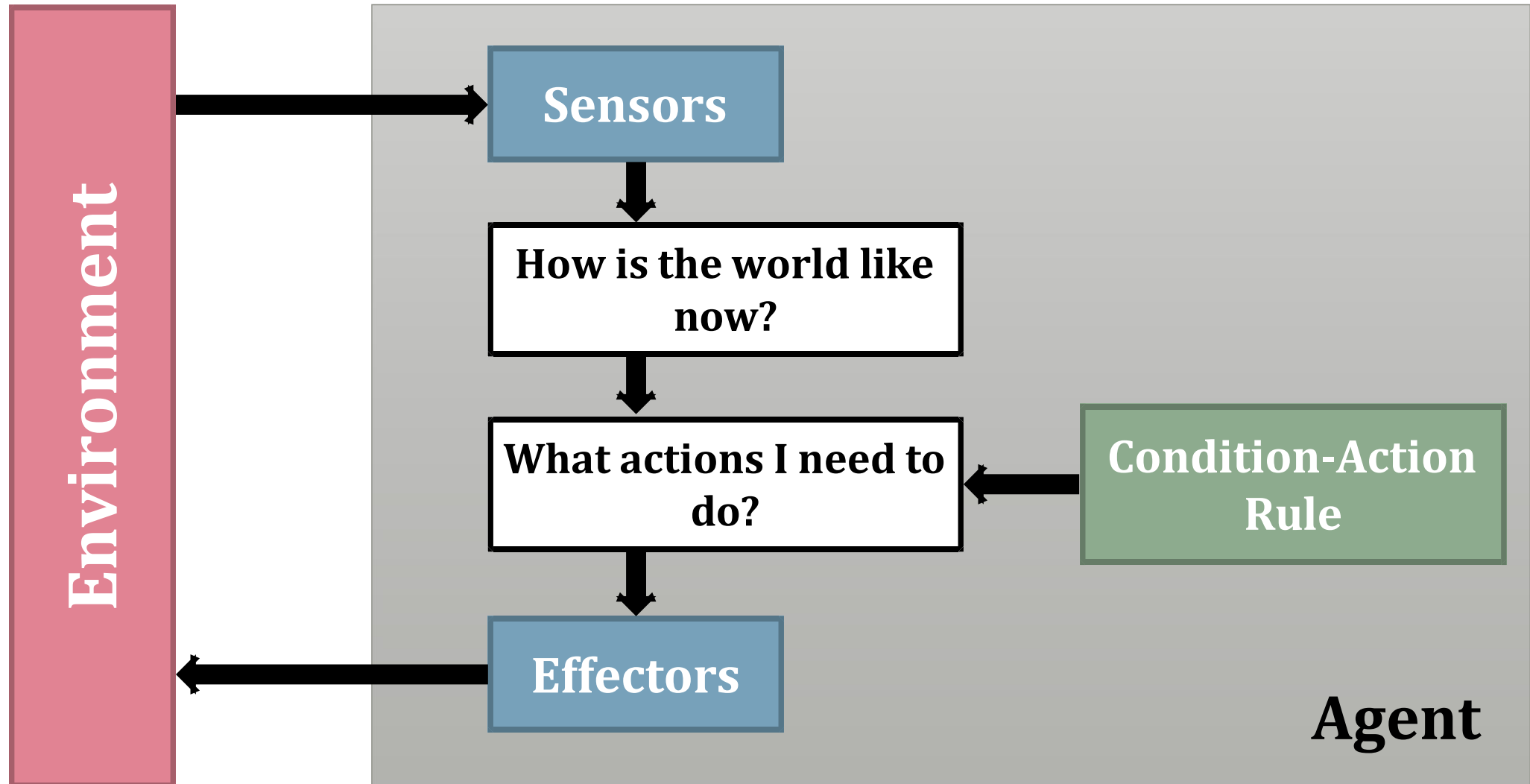
Simple Reflex Agents

- An agent whose action depends only on the current percept, ignoring the rest of the percept history.
- Rule-based reasoning to map from percepts to optimal action; each rule handles a collection of perceived states.

Problems:

- *Still usually too big to generate and to store.*
- *Still no knowledge of non-perceptual parts of state.*
- *Still not adaptive to changes in the environment; requires collection of rules to be updated if changes occur.*
- *Still can't make actions conditional on previous state.*

Simple Reflex Agents



Simple Reflex Agents

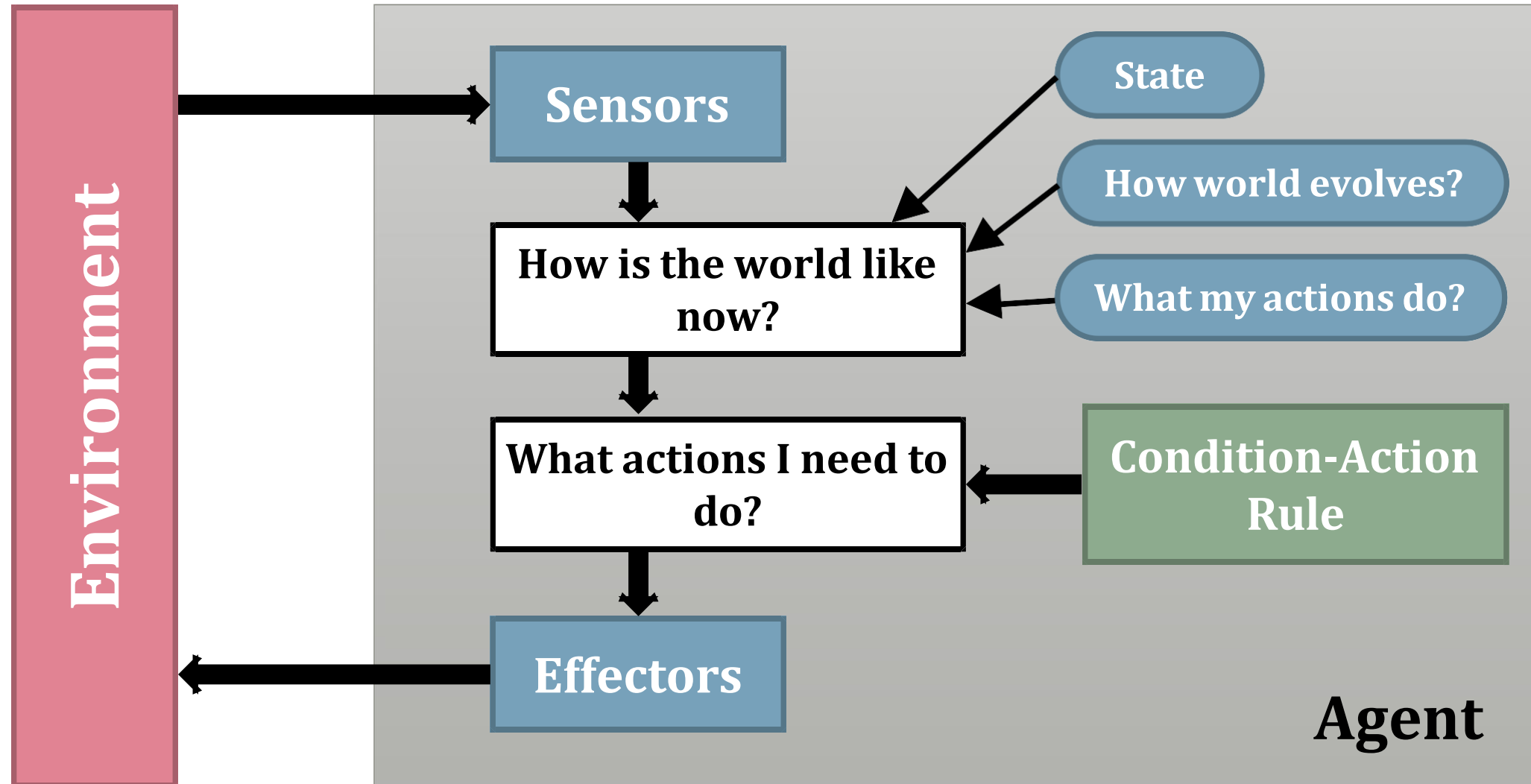
For Automated Taxi Driving Agent:

- We can generate condition-action rules.
 - *IF (car-in-front-is-braking) THEN (initiate-braking)*
 - *IF (road-is-clear) THEN (accelerate)*
 - *IF (red-signal) THEN (initiate-braking)*
 - *IF (car-in-front) AND (right-lane-clear) THEN (overtake)*

Model Based Reflex Agents

- Also called as agents with memory.
- They use a model of the world to choose their actions.
- They maintain an internal state.
- **Model** – The knowledge about how the things happen in the world.
- **Internal State** – It is a representation of unobserved aspects of current state depending on percept history.
- **Updating the state requires the information about**
 - *How the world evolves.*
 - *How the agent's actions affect the world.*

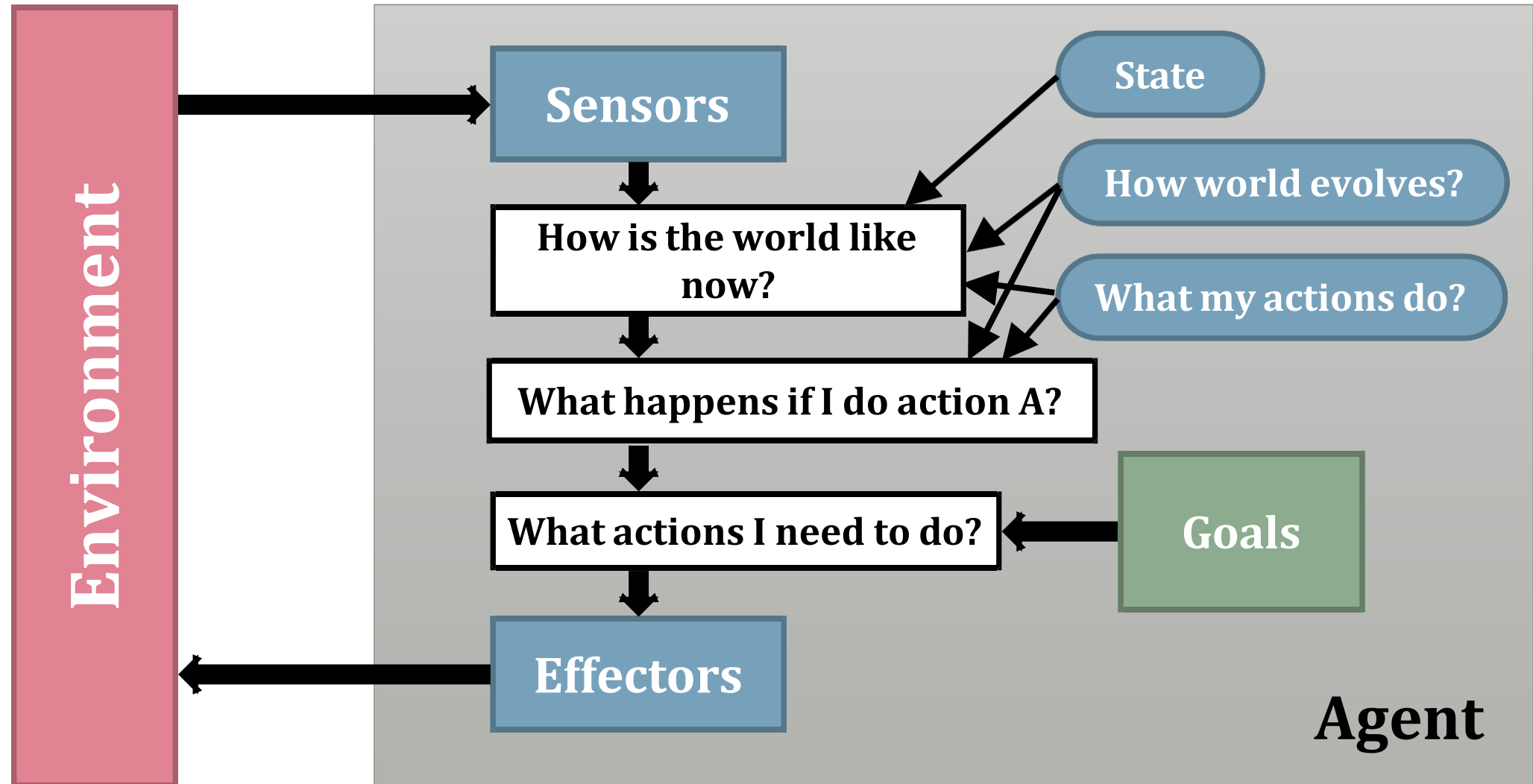
Model Based Reflex Agents



Goal Based Agents

- Choose actions so as to achieve a (given or computed) goal.
- A goal is a description of a desirable situation.
- Keeping track of the current state is often not enough, need to add goals to decide which situations are good.
- Deliberative instead of reactive.
- May have to consider long sequences of possible actions before deciding if goal is achieved – involves consideration of the future, “what will happen if I do...?”

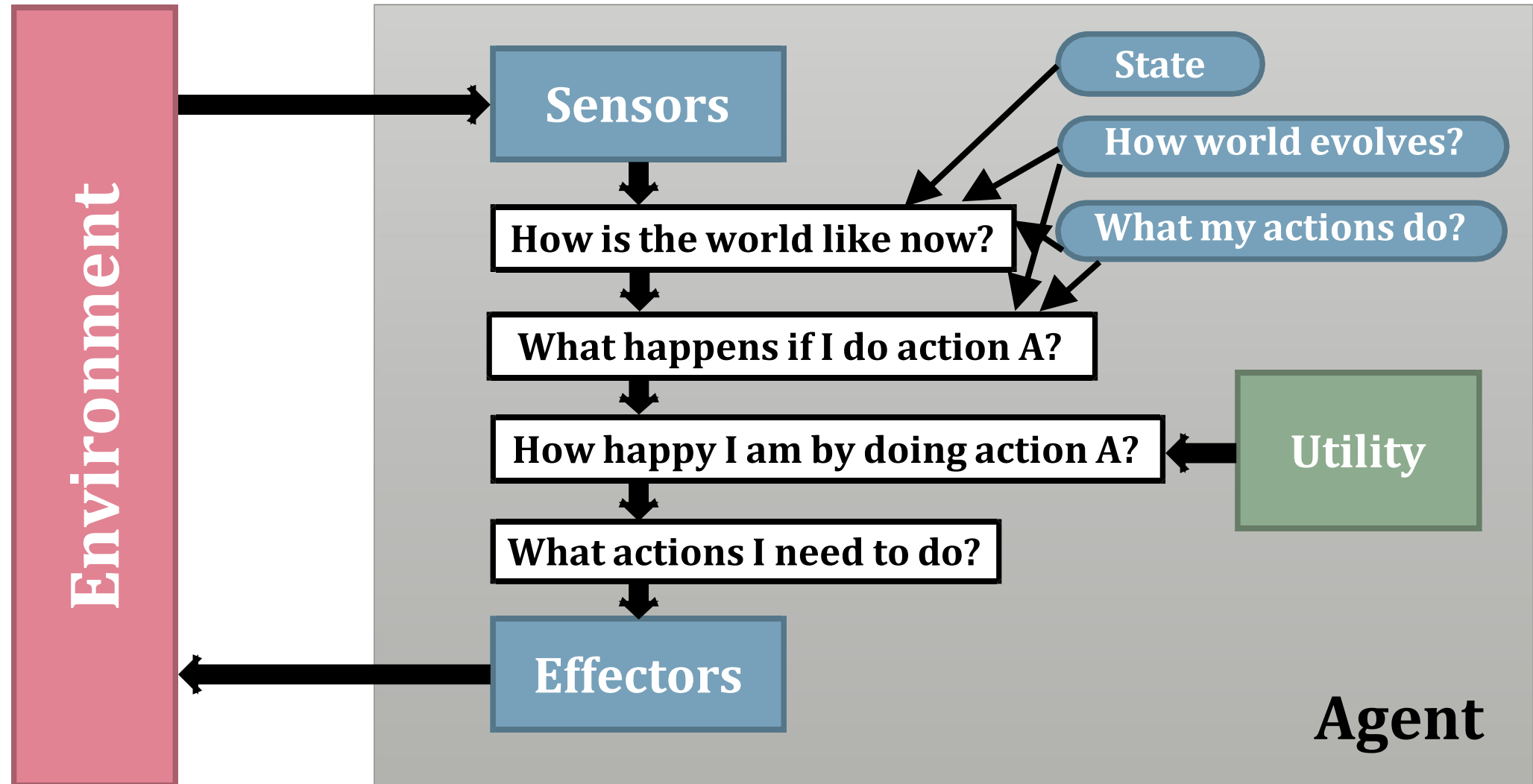
Goal Based Agents



Utility Based Agents

- Goals alone are not really enough to generate high-quality behavior.
- For example, there are many action sequences that will get the taxi to its destination, thereby achieving the goal, but some are quicker, safer, more reliable, or cheaper than others.
- Goals just provide a crude distinction between "happy" and "unhappy" states.
- Should allow a comparison of different world states, exactly how happy they would make the agent if they could be achieved.
- Because "happy" does not sound very scientific, Utility is therefore a function that maps a state onto a real number, which describes the associated degree of happiness.

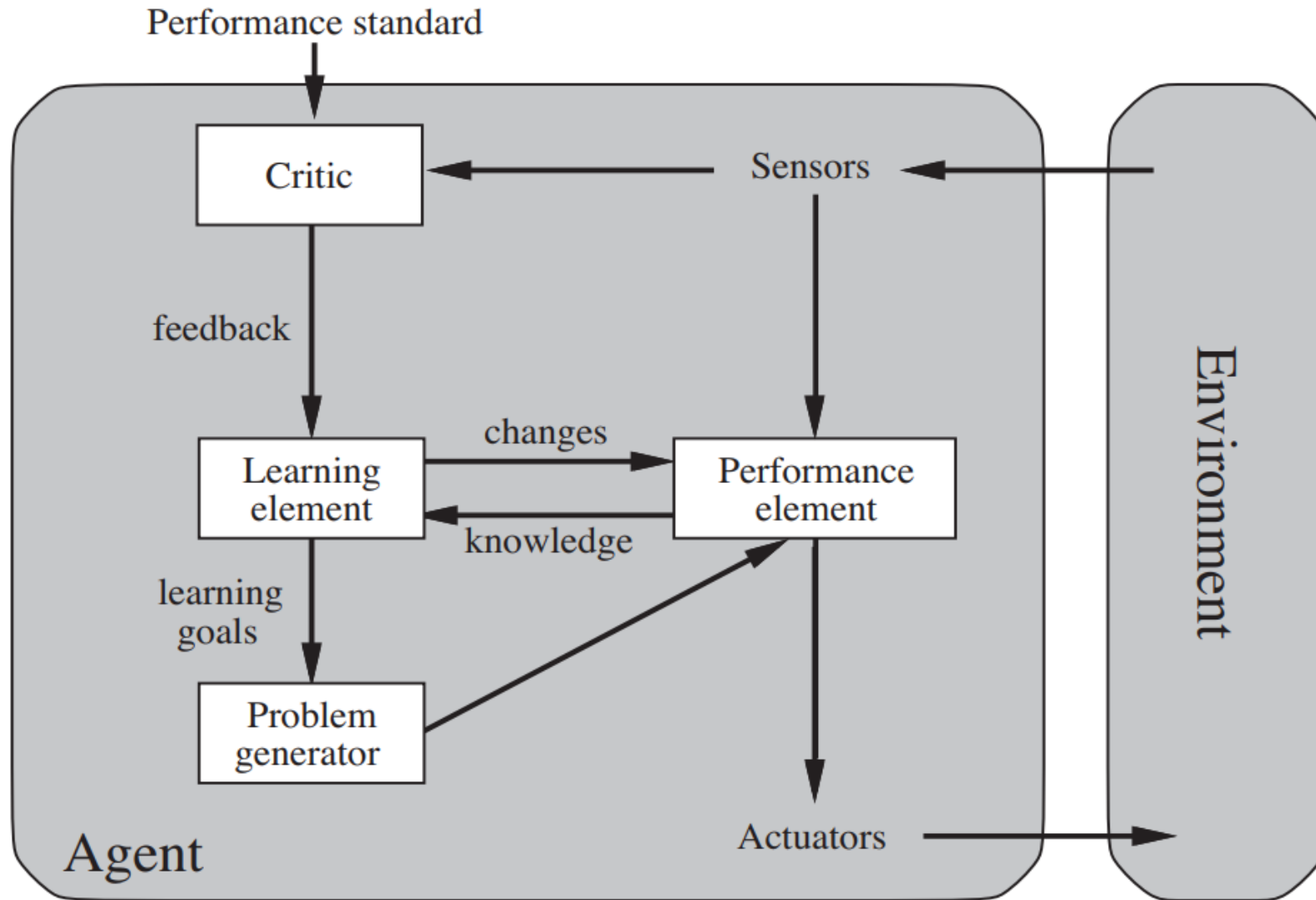
Utility Based Agents



Learning Based Agents

- It can learn from its past experience or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- Learning agents are able to learn, analyze performance, and look for new ways to improve the performance.

Learning Based Agents

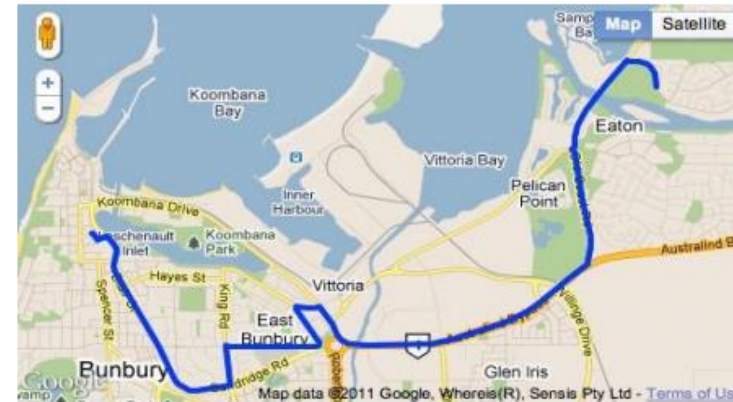


Reflex agents v. Problem solving agents

A simple reflex agent is one that selects an action based on the current percept, and ignores the rest of the percept history.



A problem-solving agent must plan ahead. It will consider a sequence of actions that form a path to a goal state. The computational process that it undertakes is called search.



Summary

- Agents interact with environments through actuators and sensors
- The agent function describes what the agent does in all circumstances.
- The performance measure evaluates the environment sequence.
- A perfectly rational agent maximizes expected performance.
- Agent programs implement (some) agent functions PEAS descriptions define task environments.
- Environments are categorized along several dimensions:
 - Observable? deterministic? episodic? static? discrete? single- agent?
- Several basic agent architectures exist:
 - Table-driven, Simple Reflex, Model based, goal-based, utility-based, Learning

Reference Material

- https://www.youtube.com/watch?v=BkedAnQfJ_U
- https://www.youtube.com/watch?v=n8z9HNV_OA
- <https://www.youtube.com/watch?v=yHXatU06h28>
- <https://www.youtube.com/watch?v=rWh9cK0ycuw>
- <https://www.youtube.com/watch?v=4GsHzPsgq-A>

Questions

1. What is an agent in the context of artificial intelligence, and how does it differ from traditional software systems?
2. Discuss the characteristics of an intelligent agent and its ability to perceive, reason, and act autonomously.
3. Define the concept of rationality in the context of intelligent agents.
4. How do rational agents make decisions to achieve their goals in dynamic and uncertain environments?
5. Explain the PEAS (Performance measure, Environment, Actuators, Sensors) framework for designing intelligent agents.
6. How does the PEAS framework help in understanding the behavior and performance of an agent in its environment?
7. Define the concept of the environment in the context of intelligent agents.
8. Discuss the role of the environment in shaping the behavior and interaction of agents.
9. Describe the different types of environments encountered by intelligent agents, such as fully observable, partially observable, deterministic, stochastic, discrete, and continuous.
10. How do different types of environments influence the design and behavior of intelligent agents?
11. What are the basic types of agents, such as reflex agents, model-based agents, goal-based agents, and utility-based agents?
12. Discuss the strengths and weaknesses of each type of agent in different environments and problem domains.

Programming Task

Designing an Intelligent Agent for Vacuum Cleaning

Description: Implement an intelligent agent that autonomously cleans a room using a vacuum cleaner. The agent should perceive its environment, make decisions based on its observations, and take actions to clean the room effectively. The task involves designing the agent's architecture, defining its rational behavior, specifying the environment, and selecting appropriate actuators and sensors.

Programming Steps:

1. **Define the Environment:**
 1. Create a simulated environment representing a room with multiple locations (e.g., squares or cells).
 2. Populate the environment with dirt particles at random locations to simulate a dirty room.
 3. Implement functions to represent the state of the environment, including the presence of dirt in each location.
2. **Design the Agent:**
 1. Define the PEAS (Performance measure, Environment, Actuators, Sensors) specifications for the agent.
 2. Choose appropriate actuators (actions) for the agent, such as moving left, moving right, sucking dirt, etc.
 3. Implement sensors to perceive the environment, such as detecting dirt in the current location.
 4. Design the agent's decision-making process to select actions based on its perceptions and rationality.
3. **Implement the Agent:**
 1. Create a class or module for the intelligent agent, encapsulating its perception, decision-making, and action-taking processes.
 2. Implement methods for the agent to perceive the environment, make decisions based on its rationality, and take appropriate actions.
 3. Test the agent's functionality in different scenarios, including varying room configurations and dirt distributions.
4. **Evaluate Performance:**
 1. Define performance metrics to evaluate the agent's effectiveness in cleaning the room, such as the percentage of dirt cleaned, time taken, and energy consumption.
 2. Run experiments to assess the agent's performance under different conditions and compare it to alternative cleaning strategies.
5. **Refine and Optimize:**
 1. Iterate on the agent's design and implementation to improve its performance and efficiency.
 2. Experiment with different rationality criteria, environment configurations, and agent architectures to optimize the cleaning process.